

Your tests pass. That is not the same as your tests working.

TestGuard takes the promises your code makes, breaks each one on purpose, and reports every promise your tests did not notice breaking.

THE PROBLEM

Coverage tells you a line ran. It never tells you anyone checked the result. So a codebase can be fully covered and completely undefended — and nothing in CI will say a word.

A REAL TEST, FROM THIS PROJECT'S OWN FIXTURE

```
expect(store.writeAudit).toHaveBeenCalledWith(  
  expect.objectContaining({ action: 'MASK', scope: 'g1', ruleCount: 1 }),  
); // `content` is never named — so nothing checks it
```

`objectContaining` ignores keys it does not list. Swap the redacted text for the **raw secret** and this test still passes. Coverage of that line: 100%. The audit log now leaks the very thing it exists to protect.

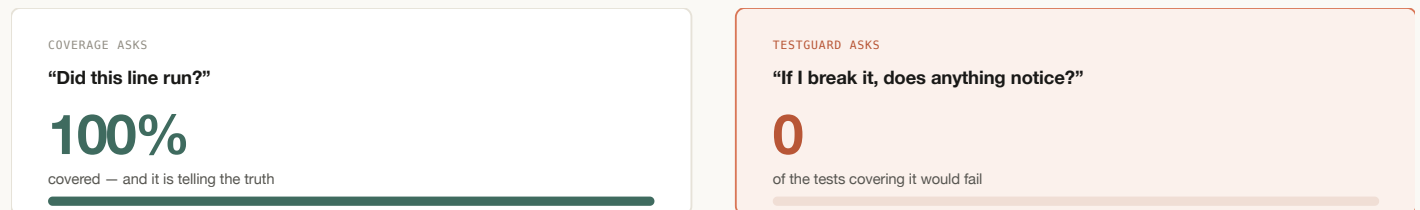


FIG. 1 — THE SAME LINE OF CODE, UNDER TWO DIFFERENT QUESTIONS. ONLY ONE OF THEM IS WORTH KNOWING.

THE METHOD

killed

You broke it — a test failed. **Good.** That behaviour is genuinely defended.

SURVIVED

You broke it — everything stayed green. **A blind spot.**

The word trips everyone once: it is the *fault* that survived, not the test. SURVIVED is the bad news. A healthy report is full of *killed*.

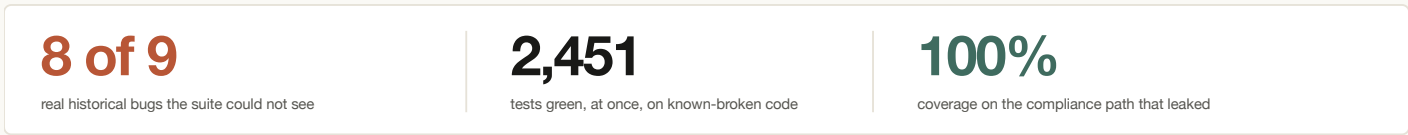
A tool that reports everything as caught is worse than no tool at all — because nobody questions good news.

THE DESIGN CONSTRAINT EVERYTHING ELSE FOLLOWS FROM

TWO INDEPENDENT RUNS, TWO AI-AUTHORED CODEBASES

Neither suite was sloppy. One had ~4,900 disciplined tests, no snapshots, and 0.4% zero-assertion. Both were green. Both were substantially blind.

FIELD REPORT A — AI-AUTHORED PRODUCTION CODEBASE, ~4,900 TESTS



FIELD REPORT B — A DIFFERENT CODEBASE, 458 TESTS, 24 SECURITY CLAIMS, 39 FAULTS INJECTED



● KILLED — DEFENDED ● SURVIVED — A BLIND SPOT

FIG. 2 — MEASURED, NOT MODELLED. EACH DOT IS ONE INJECTED FAULT.

THE SHAPE OF THE FAILURE

It is not missing tests

One file had re-implemented the authorization logic *inside the test* and asserted against its own copy. Fifteen tests, all green, all passing for the wrong reason — the production path could be deleted outright without one of them noticing.

It is assertions that name too little

The largest gap ran through a compliance-critical path at 100% coverage, where the one assertion that mattered used `objectContaining` and omitted the field carrying the data. Exactly the exhibit on page one.

A green suite is evidence that the tests agree with the code. It is not evidence that either one is right.

WHY THE QUESTION HAS TO BE ASKED ADVERSARIALLY

ONE FAULT, START TO VERDICT

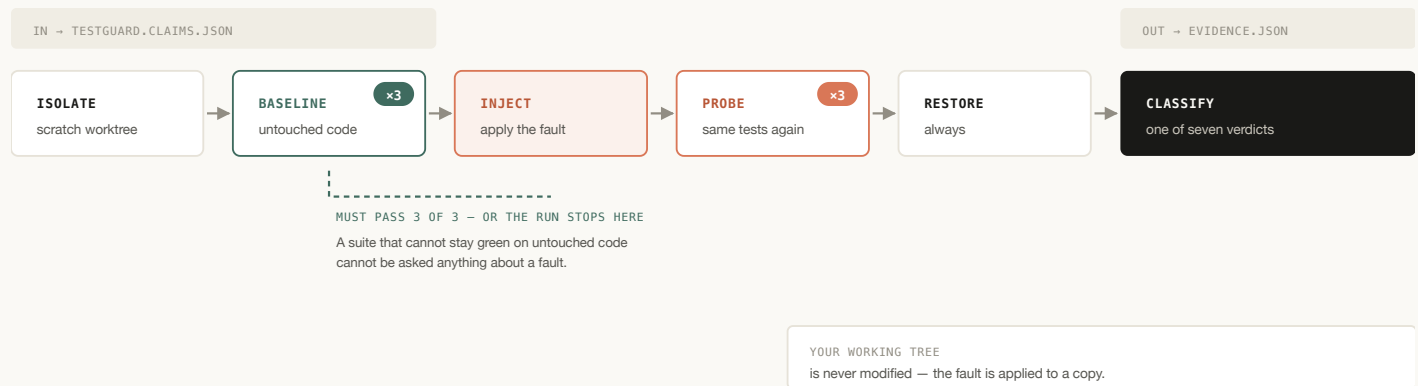
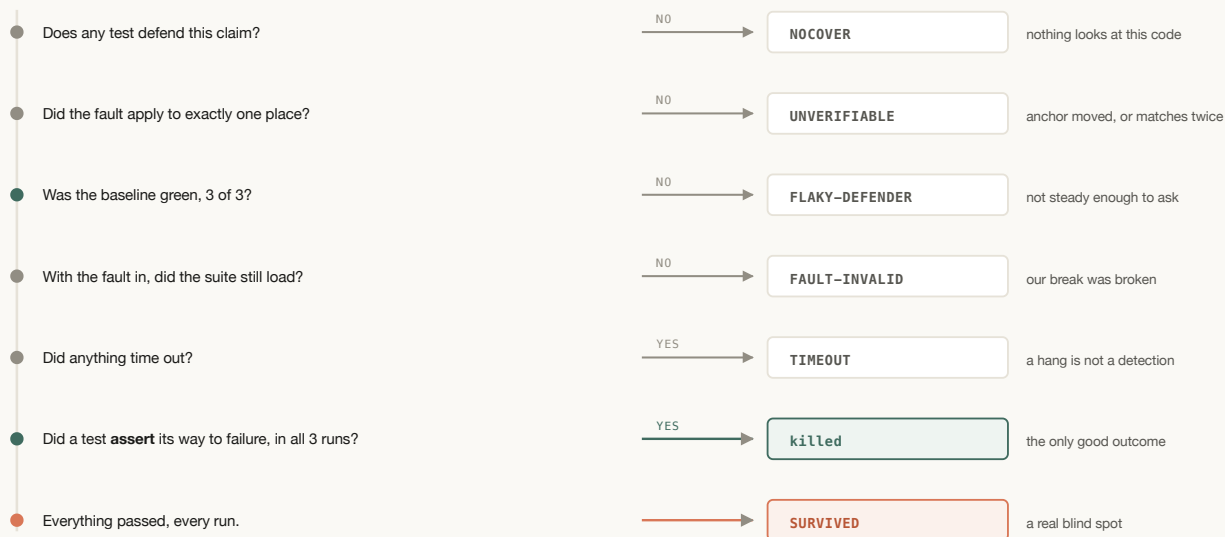


FIG. 3 — THE PROBE LOOP. EVERY STEP REPEATS PER FAULT; THE BASELINE IS CACHED PER DEFENDER SET.

HOW A VERDICT IS DECIDED

The order is the specification. Each check short-circuits the ones below it, so the tool never reaches an optimistic answer through a question it should not have asked.



ANYTHING IN BETWEEN — SOME RUNS KILLED, SOME PASSED — IS FLAKY-DEFENDER, NEVER A KILL.

FIG. 4 — VERDICT ORDER. EVERY AMBIGUITY RESOLVES DOWNWARD, TOWARD "UNPROVEN".

WHAT A RUN COSTS, AND WHY IT IS PREDICTABLE

Wall clock is not a mystery. For each claim:

$$(\text{baseline runs} + \text{faults} \times N) \times \text{cost of the claim's defender set}$$

Measured, not estimated: `testguard.cost-budget.json` records what each set actually cost in CI, and `--cost` reports the bill per claim and per file.

The defender set is the unit, not the file — a run executes all of a claim's tests at once. So one slow acceptance test named by five claims is paid for thirty times over, and the report names that file rather than leaving you to guess.

Baselines are cached per defender set, and a verdict is reused when the source, the defenders *and* the fault are all unchanged.

SEVEN VERDICTS, BECAUSE “DID THE TESTS FAIL?” IS A SLOPPY QUESTION

VERDICT	WHAT IT MEANS
killed	A test body ran and rejected the behaviour. The only good outcome.
SURVIVED	Everything passed. A real blind spot in the tests.
NOCOVER	No test even looks at this code. Not “weak tests” — <i>no</i> tests.
UNVERIFIABLE	The fault could not be applied: its anchor moved, or matches twice. Nothing was learned.
FAULT-INVALID	The break itself was broken — it did not compile. Our fault, not yours.
TIMEOUT	The suite hung. A hang is not a detection.
FLAKY-DEFENDER	The tests are not reliable enough on untouched code to be asked the question.

WHERE THE PIECES SIT

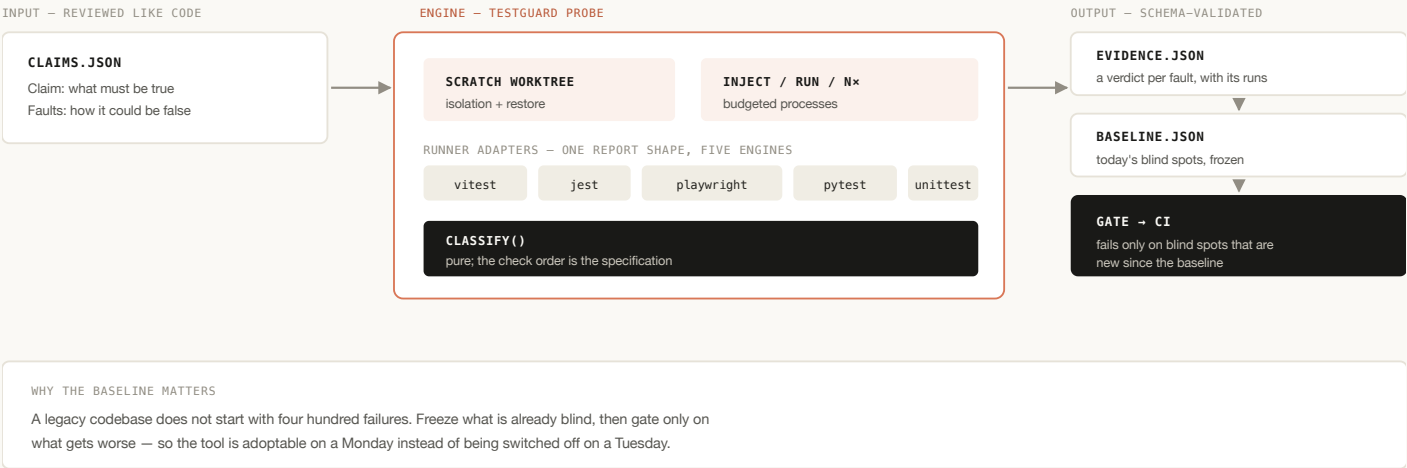


FIG. 5 – CLAIMS IN, EVIDENCE OUT, AND A GATE THAT MEASURES THE DELTA RATHER THAN THE DEBT.

EVERY AMBIGUITY ROUNDS TOWARD “UNPROVEN”

- Three runs, never one.
 - A green baseline before any fault is injected.
- A timeout is never a kill.
 - A suite that fails to load is never a kill.
- Mixed results across runs are not a kill.
 - A test that mocks what it defends is not a defender.

THE ENGINE IS ONE VERB. THE LOOP IS THE PRODUCT.

Probing proves a claim. It does not tell you which claim to write next, whether a new test earns its place, or whether the agent editing your code knows any of it.

VERB	WHY IT EXISTS
status	Where the project is and the <i>one</i> next action. <code>--json</code> is the machine entry point.
init	Installs the agent layer — skill, session-start hook, AGENTS.md section — at the git root.
claims	Lists the claims and reports drift against the <code>@claim</code> annotations in the code.
scaffold	Proposes faults mechanically for one file, as a draft claims document. A starting point, not an answer.
probe	The engine. Inject each fault, run its defenders, report what survived.
admit	The two-gate rule as one verb: green on HEAD <i>and</i> failing on every fault of the claim. Otherwise the test is not a defender.
baseline	Freezes today's unproven findings so only new ones gate.
gate	Fails when a changed source file carries no claim and no excusing entry. Unclaimed code is the blind spot before the blind test is.
brief	Emits the blind-spot block for an agent's session-start context.
mcp	Serves status, brief, claims and evidence read-only over MCP, so the loop works in any agent harness.
replay	Would this suite have caught the bugs that already escaped? <i>Reports, never gates.</i>

THE HONEST QUESTION: DOES AN INJECTED FAULT RESEMBLE A REAL BUG?

Every mutation tool has to answer this and most decline to. `replay` answers it empirically: it walks real fix commits out of your history, reverts each fix, runs the tests that exist *now*, and records whether they notice.

CALIBRATION – A MISS RATE, SO A HIGH P IS BAD

guard-removed	missed 7/9	p=0.78	ci [0.45, 0.94]
field-dropped	missed 4/4	p=1.00	ci [0.51, 1.00]

Read as: *when a real bug of this class escapes, how often does the suite miss it?* Each bug is labelled with the fault class its diff most resembles — the join key that turns an uninterpretable score into a statement with a sample size. Every `p` and interval is recomputable from `n`, and the validator recomputes them.

THE OPEN QUESTION THIS INSTRUMENT EXISTS TO ANSWER

Does a calibration learned on a repository *with* history transfer to a greenfield one that has none? AI-authored code has no history, so calibration is the only bridge. **That transfer is unproven.** It is the core product bet, and this is the instrument for testing it — not the answer.

- **Measurements enter the ratio:** `caught`, `blind`, and `nocover` — the last as a miss, because no tests at all must not score better than weak ones.
- **Failed measurements enter neither side:** `flaky` and `unverifiable` conclude nothing, and are not allowed to flatter the number.
- **Flake is biased pessimistically:** a flaky failure would read as detection, so mixed runs are never `caught`.

WHAT IS NOT NEW

Breaking code to test your tests is **mutation testing**, and it dates to the 1970s. Stryker, PIT, mutmut and Cosmic Ray all do it. Anyone claiming that part is novel is selling something.

WHAT IS DIFFERENT: THE QUESTION BEING ASKED

CLASSIC MUTATION TESTING Mutates everything, mechanically "How good are my tests?" → Mutation score: 73% A number. Not actionable, not auditable, and it cannot tell you which promise is at risk.	TESTGUARD Binds every fault to a stated claim "Is <i>this specific promise</i> defended?" → "Your project says a missing scope fails closed. Nothing checks that." A finding. An assessor reads your claims — reviewable like code — not a score.
---	--

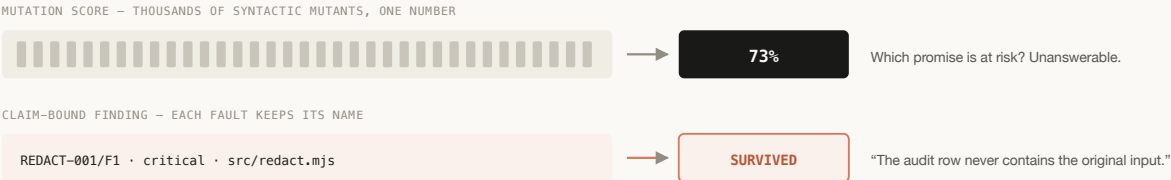


FIG. 6 — A SCORE COMPRESSES EVERYTHING INTO ONE NUMBER; A CLAIM KEEPS THE FINDING ATTRIBUTABLE.

That is the difference between a **metric** and an **audit**, and it is the whole design.

THREE MORE THINGS THAT FOLLOW FROM IT

Paranoid about itself

Most tools mark a mutant killed when the test command exits non-zero — conflating “a test caught it” with “everything exploded.” Here the verdict set is closed and each member means something different.

Evidence, not output

Schema-validated JSON with stable fingerprints, shared with the other Guard tools. Freeze today’s blind spots, gate CI on *new* ones.

Built for AI-written code

When an agent writes the tests, the failure mode is not “no tests.” It is confident, plausible tests that assert nothing. That is precisely what this detects.

Coverage tells you a line ran. TestGuard tells you which of your promises nobody is actually defending.

GITHUB.COM/RACCIOLY/TESTGUARD · NPM TESTGUARD-CLI · MIT